

다음 GitHub 프로젝트의 백엔드 API를 구현해줘.

Repository: <https://github.com/jeongdongin050416-star/timetable-recommender>

작업 대상:

- backend 디렉터리
- Java 21
- Spring Boot
- Gradle
- Spring Data JPA
- PostgreSQL
- Flyway
- Spring Validation

먼저 현재 프로젝트의 엔티티, Flyway 마이그레이션, Repository, 패키지 구조, SecurityConfig, 테스트 설정을 전부 확인해라.

기존 코드를 임의로 제거하거나 현재 DB 구조와 충돌하는 새 구조를 만들지 말고, 기존 설계에 맞춰 확장해라.

작업을 시작하기 전에 현재 구조와 구현 계획을 간단히 정리한 뒤, 별도의 확인을 기다리지 말고 구현까지 진행해라.

구현 목표

다음 기능을 구현한다.

1. 사용자별 이수 과목 조회
2. 사용자별 이수 과목 추가
3. 사용자별 이수 과목 삭제
4. 사용자 조건에 따른 추천 과목 조회
5. 회원가입 및 로그인
6. 일관된 API 응답 형식
7. 전역 예외 처리
8. Controller 통합 테스트
9. API 문서 및 curl 예제 작성

공통 개발 규칙

다음 규칙을 반드시 지켜라.

1. JPA Entity를 Controller에서 직접 요청 또는 응답 객체로 사용하지 않는다.
2. 모든 API에 별도의 Request DTO와 Response DTO를 사용한다.
3. Controller에는 비즈니스 로직을 넣지 않는다.
4. Controller는 요청 검증, Service 호출, 응답 반환만 담당한다.
5. 비즈니스 로직과 트랜잭션 경계는 Service 계층에 둔다.
6. Entity를 JSON으로 직접 직렬화하지 않는다.
7. DTO는 Java record를 우선 사용하되, 기존 프로젝트 스타일과 충돌하면 일반 클래스를 사용한다.
8. Lombok은 추가하지 않는다.

9. 조회 작업에는 적절한 경우 `@Transactional(readOnly = true)` 를 사용한다.
10. 데이터 변경 작업에는 `@Transactional` 을 사용한다.
11. 예외를 Controller에서 개별적으로 try-catch 하지 않는다.
12. `@RestControllerAdvice` 를 사용해 예외를 공통 처리한다.
13. 사용자가 존재하지 않으면 404를 반환한다.
14. 과목이 존재하지 않으면 404를 반환한다.
15. 요청에서 지정한 관심 분야가 존재하지 않으면 404를 반환한다.
16. 잘못된 요청 값은 400을 반환한다.
17. 비밀번호 오류 등 로그인 실패는 401을 반환한다.
18. 이메일 중복과 같은 리소스 충돌은 409를 반환한다.
19. 예외 응답에 stack trace, SQL, 내부 클래스명 등을 노출하지 않는다.
20. 테스트를 통과시키기 위해 운영 코드를 부자연스럽게 변경하지 않는다.
21. 기존 Flyway migration 파일을 수정하지 않는다. DB 변경이 필요하면 새로운 migration 파일을 추가한다.
22. Repository와 Service에서 불필요한 전체 테이블 조회나 N+1 문제가 발생하지 않도록 한다.
23. 결과 순서가 매 실행마다 달라지지 않도록 정렬 기준을 명시한다.
24. 현재 단계에서는 프론트엔드를 수정하지 않는다.

공통 응답 형식

성공과 실패를 포함해 모든 API가 동일한 최상위 JSON 구조를 사용하도록 구현한다.

성공 응답 예시:

```
{
  "success": true,
  "data": {
    "userId": 1
  },
  "error": null
}
```

실패 응답 예시:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "USER_NOT_FOUND",
    "message": "사용자를 찾을 수 없습니다.",
    "fieldErrors": null
  }
}
```

Validation 실패 예시:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "INVALID_REQUEST",
    "message": "요청 값이 올바르지 않습니다.",
    "fieldErrors": {
      "courseCount": "1 이상이어야 합니다."
    }
  }
}
```

다음과 유사한 공통 객체를 구현한다.

- ApiResponse<T>
- ApiError
- 필요하면 FieldErrorResponse

응답 객체에 Entity를 포함해서는 안 된다.

기존 GET /api/health 도 가능하면 같은 응답 형식으로 변경하고, 관련 문서와 테스트를 함께 수정한다.

1. 이수 과목 조회 API

Endpoint

```
GET /api/users/{userId}/completed-courses
```

동작

- 사용자가 존재하는지 먼저 확인한다.
- 사용자가 존재하지 않으면 404 USER_NOT_FOUND
- 사용자가 이수한 과목들을 조회한다.
- 이수 과목이 없으면 빈 배열을 반환한다.
- 과목 코드를 기준으로 오름차순 정렬한다.
- Entity를 반환하지 않고 Response DTO로 변환한다.

응답 데이터 예시

```
{
  "success": true,
  "data": {
    "userId": 1,
    "courses": [
      {
```

```

    "courseId": 10,
    "courseCode": "CS101",
    "name": "프로그래밍 기초",
    "credits": 3
  }
],
"error": null
}

```

현재 Course Entity의 필드명이 위 예시와 다르면 기존 필드에 맞게 DTO 필드를 조정하되, 최소한 다음 정보는 포함한다.

- courseId
- courseCode
- 과목명
- 학점: 현재 스키마에 존재하는 경우

2. 이수 과목 추가 API

Endpoint

```
PUT /api/users/{userId}/completed-courses/{courseId}
```

동작

- 사용자와 과목의 존재 여부를 각각 확인한다.
- 사용자가 없으면 404 `USER_NOT_FOUND`
- 과목이 없으면 404 `COURSE_NOT_FOUND`
- 아직 등록되지 않은 과목이면 `completed_course`를 생성한다.
- 이미 등록된 과목이면 아무것도 추가하지 않고 성공 처리한다.
- 동일 요청을 여러 번 실행해도 최종 상태가 같아야 한다.
- DB unique constraint와 애플리케이션 로직을 모두 고려한다.
- 동시 요청에서도 중복 데이터가 생성되지 않도록 기존 unique constraint를 확인한다.
- 성공 시 항상 200을 반환한다.

응답 데이터 예시

```

{
  "success": true,
  "data": {
    "userId": 1,
    "courseId": 10,
    "completed": true
  },
}

```

```
"error": null
}
```

이미 등록된 상태인지 새로 등록한 상태인지는 API 사용자가 반드시 알 필요가 없으므로 동일한 성공 응답을 반환한다.

3. 이수 과목 삭제 API

Endpoint

```
DELETE /api/users/{userId}/completed-courses/{courseId}
```

동작

- 사용자와 과목의 존재 여부를 각각 확인한다.
- 사용자가 없으면 404 `USER_NOT_FOUND`
- 과목이 없으면 404 `COURSE_NOT_FOUND`
- `completed_course` 관계가 있으면 삭제한다.
- 이미 삭제되어 관계가 없더라도 성공 처리한다.
- 동일 요청을 여러 번 실행해도 최종 상태가 같아야 한다.
- 성공 시 항상 200을 반환한다.

응답 데이터 예시

```
{
  "success": true,
  "data": {
    "userId": 1,
    "courseId": 10,
    "completed": false
  },
  "error": null
}
```

4. 추천 과목 조회 API

Endpoint

GET 요청 body는 사용하지 말고 query parameter를 사용한다.

```
GET /api/users/{userId}/recommended-courses
```

Query parameters:

```
courseCount=5
interestedAreaIds=1,2
excludedAreaIds=3,4
```

Spring MVC에서 List<Long>으로 자연스럽게 받을 수 있는 형식을 사용한다. 쉼표 구분 또는 반복 파라미터 중 하나를 선택하고 API 문서와 curl 예제에서 동일한 방식을 사용한다.

요청 예시

```
GET /api/users/1/recommended-courses?
courseCount=5&interestedAreaIds=1,2&excludedAreaIds=3
```

Validation

- `courseCount` 는 필수
- `courseCount` 는 1 이상
- 현실적인 상한을 설정한다. 상한은 20으로 한다.
- 관심 분야와 제외 분야는 생략할 수 있다.
- 같은 분야 ID가 관심 분야와 제외 분야에 동시에 들어오면 400 `CONFLICTING_INTEREST_AREA`
- 존재하지 않는 관심 분야 ID가 있으면 404 `INTEREST_AREA_NOT_FOUND`
- 중복으로 전달된 분야 ID는 한 번만 처리한다.

추천 로직

이번 API는 전체 시간표 생성 API가 아니라 과목 추천 API다. 이번 작업에서 시간표 조합 생성이나 분반 시간 충돌 계산 까지 확장하지 않는다.

현재 DB와 Entity 관계를 확인한 뒤 다음 기준으로 추천한다.

반드시 적용할 조건

1. 사용자가 이미 이수한 과목은 제외한다.
2. 제외 분야에 속하는 과목은 제외한다.
3. 선수 과목 관계가 구현되어 있다면, 필요한 선수 과목을 사용자가 모두 이수한 과목만 추천한다.
4. 동일한 과목이 중복 반환되지 않게 한다.
5. 요청한 `courseCount` 까지만 반환한다.
6. 조건을 만족하는 과목이 부족하면 가능한 과목만 반환하고 오류를 발생시키지 않는다.

우선순위

1. 사용자의 관심 분야와 일치하는 개수가 많은 과목
2. 전공필수 여부가 현재 스키마에 존재한다면 전공필수 과목
3. 과목 코드 오름차순

점수 계산을 별도 메서드나 정책 클래스로 분리해 읽기 쉽게 구현한다.

예시:

관심 분야 일치 1개당 높은 가중치 부여
전공필수 과목에 추가 가중치 부여
최종 점수 내림차순
동점이면 courseCode 오름차순

매직 넘버를 Service 내부에 흩뿌리지 말고 상수 또는 추천 정책 클래스로 관리한다.

응답 데이터 예시

```
{
  "success": true,
  "data": {
    "userId": 1,
    "requestedCourseCount": 5,
    "returnedCourseCount": 3,
    "courses": [
      {
        "courseId": 20,
        "courseCode": "CS201",
        "name": "데이터 구조",
        "credits": 3,
        "score": 210,
        "interestAreas": [
          {
            "interestAreaId": 1,
            "name": "백엔드"
          }
        ],
        "reasons": [
          "관심 분야 1개 일치",
          "전공필수 과목"
        ]
      }
    ]
  },
  "error": null
}
```

현재 Entity와 스키마에 없는 필드는 억지로 추가하지 않는다. 다만 추천 이유를 응답할 수 있도록 Service에서 계산한 `score`, `reasons` 는 Response DTO에 포함해도 된다.

5. 회원가입 API

로그인 테스트와 실제 사용을 위해 회원가입 API도 함께 구현한다.

Endpoint

POST /api/auth/signup

Request 예시

```
{
  "email": "user@example.com",
  "password": "password123!",
  "name": "홍길동"
}
```

현재 AppUser Entity가 email 대신 username 등 다른 로그인 식별자를 사용한다면 기존 구조를 존중하되, API 전체에서 하나의 로그인 식별자 방식을 일관되게 사용한다.

Validation

- 로그인 식별자는 필수
- 이메일 방식을 사용한다면 이메일 형식 검증
- 비밀번호 필수
- 비밀번호 최소 8자
- 이름 필수
- 중복 이메일 또는 중복 로그인 ID는 409 `DUPLICATE_USER`

비밀번호 처리

- 비밀번호를 평문으로 DB에 저장하지 않는다.
- BCrypt로 해시한 값만 저장한다.
- 응답에 password 또는 passwordHash를 절대 포함하지 않는다.
- 로그에도 비밀번호를 출력하지 않는다.
- 기존 app_user 테이블에 비밀번호 해시 컬럼이 없다면 기존 migration을 수정하지 말고 새로운 Flyway migration을 추가한다.
- 필요한 unique constraint 역시 새 migration으로 추가한다.
- Spring Security 전체 인증 기능을 활성화하지 않더라도 BCrypt 기반 PasswordEncoder는 사용할 수 있다.

성공 응답

- HTTP 201 Created

```
{
  "success": true,
  "data": {
    "userId": 1,
    "email": "user@example.com",
    "name": "홍길동"
  },
}
```

```
"error": null
}
```

6. 로그인 API

Endpoint

```
POST /api/auth/login
```

Request 예시

```
{
  "email": "user@example.com",
  "password": "password123!"
}
```

동작

- 사용자 조회
- BCrypt를 이용한 비밀번호 일치 여부 확인
- 이메일이 존재하지 않거나 비밀번호가 틀리면 동일하게 401 `INVALID_CREDENTIALS`
- 어떤 값이 틀렸는지 구분해서 알려주지 않는다.
- 성공하면 사용자 기본 정보를 반환한다.
- passwordHash는 절대 반환하지 않는다.

성공 응답

```
{
  "success": true,
  "data": {
    "userId": 1,
    "email": "user@example.com",
    "name": "홍길동"
  },
  "error": null
}
```

현재 인증 범위 제한

이번 작업에서는 다음을 구현하지 않는다.

- JWT 발급
- Access Token

- Refresh Token
- 로그인 세션
- 쿠키 인증
- 인증 필터
- API별 접근 권한 확인
- SecurityContext 사용자 연동
- 현재 로그인한 사용자를 이용한 userId 자동 추출

현재 SecurityConfig는 모든 요청을 허용하는 상태를 유지한다.

즉, 이번 로그인 API는 사용자 자격 증명을 검증하는 API까지만 구현한다. 이후 별도 작업에서 JWT 또는 세션 기반 인증을 적용할 수 있도록 AuthService의 책임을 명확히 분리한다.

CSRF, formLogin, httpBasic 등 현재 설정이 API 테스트를 방해하지 않도록 기존 SecurityConfig를 확인하되, 모든 API는 계속 permitAll 상태여야 한다.

권장 패키지 구조

현재 패키지 구조를 먼저 확인하고, 기존 스타일을 우선한다. 새로운 기능은 다음과 유사하게 역할별로 분리한다.

```
com.example.timetablerecommender
├── common
│   ├── api
│   │   ├── ApiResponse
│   │   └── ApiError
│   └── exception
│       ├── GlobalExceptionHandler
│       ├── UserNotFoundException
│       ├── CourseNotFoundException
│       ├── InterestAreaNotFoundException
│       ├── InvalidCredentialsException
│       └── DuplicateUserException
├── auth
│   ├── controller
│   ├── dto
│   └── service
├── completedcourse
│   ├── controller
│   ├── dto
│   ├── service
│   └── repository
└── recommendation
    ├── controller
    ├── dto
    ├── service
    └── policy
```

기존 domain 패키지에 Entity와 Repository가 이미 있다면 무리하게 이동하지 않는다. 기존 파일 이동으로 불필요한 변경 범위를 늘리지 마라.

Repository 요구사항

기존 Repository가 있으면 확장하고, 없으면 생성한다.

필요한 기능의 예:

- 사용자 존재 여부 및 사용자 조회
- 과목 존재 여부 및 과목 조회
- 사용자별 이수 과목 조회
- 특정 사용자와 과목의 completed_course 존재 여부
- 특정 사용자와 과목의 completed_course 삭제
- 사용자의 이수 과목 ID 조회
- 관심 분야를 포함한 추천 후보 과목 조회

다음 사항을 고려한다.

- completed_course 중복 방지
- 적절한 unique constraint 활용
- 추천 조회 시 N+1 방지
- 불필요한 `findAll()` 후 Java 메모리 필터링 지양
- 필요하다면 JPQL, EntityGraph 또는 fetch join 사용
- collection fetch join과 pagination 충돌에 주의
- 추천 후보 규모가 작더라도 확장 가능한 구조 유지

예외 코드

최소한 다음 예외 코드를 사용한다.

```
USER_NOT_FOUND
COURSE_NOT_FOUND
INTEREST_AREA_NOT_FOUND
INVALID_REQUEST
CONFLICTING_INTEREST_AREA
DUPLICATE_USER
INVALID_CREDENTIALS
INTERNAL_SERVER_ERROR
```

예상하지 못한 예외는 500으로 처리하되 내부 예외 메시지를 그대로 노출하지 않는다.

Controller 통합 테스트

단순한 Controller 단위 테스트가 아니라 실제 Controller, Service, Repository, JPA 매핑, Validation, 예외 처리까지 검증하는 통합 테스트를 작성한다.

권장 방식:

- `@SpringBootTest`
- `@AutoConfigureMockMvc`
- `MockMvc`
- 실제 Service와 Repository 사용
- PostgreSQL Testcontainers 사용
- 각 테스트가 서로 영향을 주지 않도록 데이터 정리 또는 트랜잭션 rollback 적용

H2와 PostgreSQL의 SQL 문법 및 제약 조건 차이로 테스트가 실제 운영 DB와 다르게 동작하지 않도록 PostgreSQL Testcontainers를 우선 사용한다.

Testcontainers를 사용한다면 필요한 Gradle test dependency와 Spring Boot 테스트 설정을 추가한다.

Security가 적용되어 있더라도 현재 API는 `permitAll` 상태로 테스트되어야 한다.

필수 테스트 목록

공통 응답

- 성공 응답이 `success`, `data`, `error` 구조를 가진다.
- 오류 응답이 같은 최상위 구조를 가진다.
- Validation 오류가 `fieldErrors`를 반환한다.
- Entity의 내부 필드나 비밀번호 해시가 응답에 노출되지 않는다.

이수 과목 조회

- 존재하는 사용자의 이수 과목 조회 성공
- 이수 과목이 없는 사용자는 빈 배열 반환
- 과목 코드 오름차순 정렬
- 존재하지 않는 사용자 조회 시 404

이수 과목 추가

- 새로운 이수 과목 추가 성공
- 추가 후 DB에 `completed_course`가 저장됨
- 같은 PUT을 두 번 호출해도 200
- 같은 PUT을 두 번 호출해도 DB 행은 하나만 존재
- 존재하지 않는 사용자면 404
- 존재하지 않는 과목이면 404

이수 과목 삭제

- 기존 이수 과목 삭제 성공
- 삭제 후 DB에서 관계가 제거됨
- 이미 삭제된 과목을 다시 DELETE해도 200
- 관계가 원래 없던 과목을 DELETE해도 200
- 존재하지 않는 사용자면 404
- 존재하지 않는 과목이면 404

추천 과목

- 사용자가 이수한 과목이 추천 결과에서 제외됨
- 제외 분야 과목이 추천 결과에서 제외됨
- 관심 분야와 일치하는 과목이 우선됨
- 전공필수 필드가 있다면 전공필수 과목이 우선됨
- 선수 과목 관계가 있다면 미충족 과목이 제외됨
- courseCount만큼 제한됨
- 후보가 부족하면 적은 개수를 정상 반환
- 동점 결과가 과목 코드 순으로 정렬됨
- courseCount가 0이면 400
- courseCount가 20을 초과하면 400
- 관심 분야와 제외 분야가 겹치면 400
- 존재하지 않는 사용자면 404
- 존재하지 않는 관심 분야면 404

회원가입

- 회원가입 성공 시 201
- DB에 사용자가 저장됨
- 저장된 비밀번호가 평문과 다름
- BCrypt matches로 원래 비밀번호를 검증할 수 있음
- 중복 로그인 식별자로 가입하면 409
- 잘못된 이메일 또는 짧은 비밀번호면 400
- 응답에 비밀번호와 비밀번호 해시가 없음

로그인

- 올바른 자격 증명으로 로그인 성공
- 존재하지 않는 사용자면 401
- 비밀번호가 틀리면 401
- 두 실패 경우 모두 `INVALID_CREDENTIALS` 를 반환
- 응답에 비밀번호와 비밀번호 해시가 없음

API 문서

구현 완료 후 `docs/API.md` 를 업데이트한다.

각 API마다 다음 내용을 기록한다.

- HTTP method
- URL
- Path parameter
- Query parameter
- Request body
- Validation 조건
- 성공 status code
- 성공 응답 예시
- 실패 status code
- 실패 응답 예시
- 역등성 여부

특히 PUT과 DELETE가 역등하게 동작한다는 점을 문서에 명시한다.

curl 예제

작업 완료 후 최종 답변에 생성한 API별 실행 가능한 curl 예제를 모두 보여줘.

최소한 다음 예제를 포함한다.

1. Health check
2. 회원가입
3. 로그인
4. 이수 과목 목록 조회
5. 이수 과목 추가
6. 같은 이수 과목 재추가
7. 이수 과목 삭제
8. 이미 삭제된 이수 과목 재삭제
9. 관심 분야와 제외 분야를 포함한 추천 과목 조회
10. 유효성 검증 실패 예제
11. 존재하지 않는 사용자 404 예제
12. 로그인 실패 예제

Windows PowerShell이 아니라 Bash 기준 curl로 작성한다.

JSON body를 보내는 요청에는 다음 헤더를 포함한다.

```
-H "Content-Type: application/json"
```

예상 형태:

```
curl -X POST "http://localhost:8080/api/auth/signup"  
-H "Content-Type: application/json"  
-d '{  
  "email": "user@example.com",  
  "password": "password123!",  
  "name": "홍길동"  
}'
```

```
curl -X PUT  
"http://localhost:8080/api/users/1/completed-courses/10"
```

```
curl  
"http://localhost:8080/api/users/1/recommended-courses?  
courseCount=5&interestedAreaIds=1,2&excludedAreaIds=3"
```

실제 구현에서 선택한 요청 필드와 query parameter 형식에 맞게 예제를 수정한다.

구현 완료 후 검증

구현 완료 후 다음 명령을 실행해라.

```
cd backend
./gradlew clean test
```

가능하면 애플리케이션 시작 가능 여부도 확인한다.

```
./gradlew bootRun
```

DB 또는 Docker 환경 문제 때문에 실행하지 못한 검증이 있다면 성공했다고 주장하지 말고, 어떤 검증을 실행했고 어떤 검증을 실행하지 못했는지 정확히 설명한다.

최종 보고 형식

구현이 끝나면 다음 순서로 결과를 보고한다.

1. 기존 구조 분석 요약
2. 구현한 기능 요약
3. 주요 설계 결정
4. 생성하거나 수정한 파일 목록
5. API별 HTTP method와 URL
6. 응답 형식
7. 역등성 처리 방식
8. 추천 알고리즘
9. 로그인 구현 범위와 아직 구현하지 않은 인증 기능
10. 테스트 목록
11. 실제 테스트 실행 결과
12. API별 curl 예제
13. 남아 있는 기술적 제한 또는 후속 작업

코드만 생성하고 끝내지 말고, 모든 변경 사항이 서로 일관되게 동작하도록 실제 테스트까지 작성해라.